

week9 note

1. list 的几个操作效率

二、Assigning (赋值)

定义:

给变量或列表中的某个位置赋值。

示例:

```
python
1 a = 10 # 给变量 a 赋值
2 lst = [0] * 5
3 lst[2] = 99 # 给列表中索引为 2 的位置赋值
```

为什么快?

- 变量赋值在 Python 中是非常基础的操作，底层直接映射到内存地址绑定。
- 列表的按索引赋值也是 $O(1)$ 时间复杂度，非常高效。

三、List Comprehensions (列表推导式)

定义:

一种简洁的语法，用来快速生成列表。

示例:

```
python
1 squares = [x**2 for x in range(10)]
2 print(squares) # 输出: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

为什么快?

- 列表推导式的底层实现是用 C 编写的循环逻辑，避免了 Python 解释器的开销。
- 对比普通的 `for` 循环和 `append()`，列表推导式通常更快，因为其内部优化了迭代过程。

Inserting (beginning/middle), slicing, and adding lists are slow:

3	5	2	9	7	3	4	2	
---	---	---	---	---	---	---	---	--

3	5	2	7	
---	---	---	---	--

 +

9	
---	--

3	5	2	7	9	
---	---	---	---	---	--

Example: building long lists of perfect squares (numbers that are an integer times itself)

```
def using_list_comprehension(n):
    return [k * k for k in range(n)]
```

```
def using_append(n):
    s = []
    for k in range(n):
        s.append(k * k)
    return s
```

```
def using_assign(n):
    s = [0 for k in range(n)]
    for k in range(n):
        s[k] = k * k
    return s
```

```
def using_insert(n):
    s = []
    for k in range(n):
        s.insert(0, k * k)
    return s
```

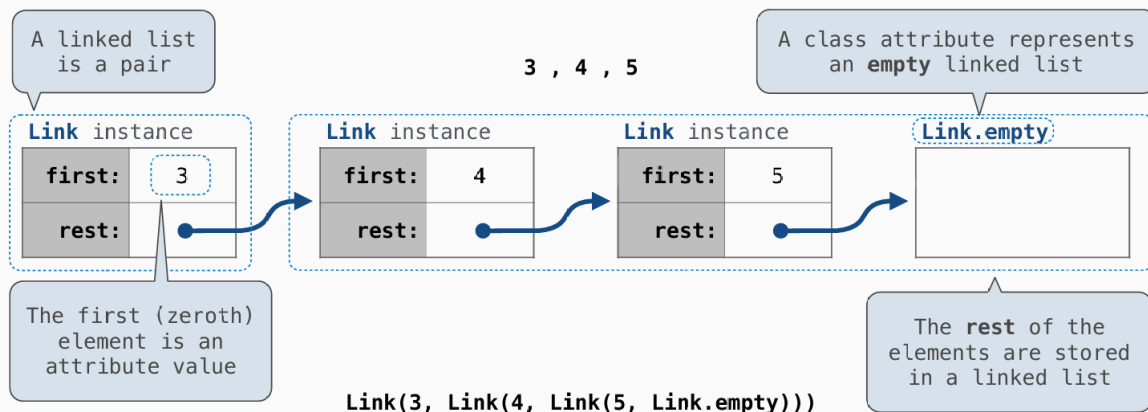
```
def using_add(n):
    s = []
    for k in range(n):
        s = s + [k*k]
    return s
```

When $n = 100,000$

using_list_comprehension:	1.58 ms
using_append:	1.76 ms
using_assign:	2.58 ms
using_insert:	1,470 ms
using_add:	9,210 ms

2. Linked List Structure

A linked list is either empty or a first value and the rest of the linked list



Linked list class: attributes are passed to `__init__`

```
class Link:
    empty = ()
    def __init__(self, first, rest=empty):
        assert rest is Link.empty or isinstance(rest, Link)
        self.first = first
        self.rest = rest
```

Some zero-length sequence

Returns whether rest is a Link

这个是数据结构，具体的类

3. 注释和help函数调用的关系

当你在一个函数中写上这样的 文档字符串 (docstring) :

```
python
1 def double(s, v):
2     """Insert another v after each v in list s.
3     >>> s = [2, 7, 1, 8, 2, 8]
4     >>> double(s, 8)
5     >>> s
6     [2, 7, 1, 8, 8, 2, 8, 8]
7     """
8     ...
```

然后在 Python shell 中使用 `help(double)` 或者 `help(your_module.double)` , 这段字符串就会被显示出来, 作为函数的说明文档。

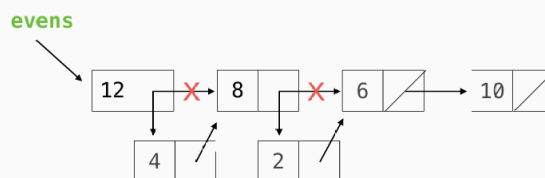
是docstring

4. 下面是一个分割链表的操作，我简直为此而折服，不管效率的话，真的简洁

5. 链表的插入操作

```
def insert_link(s, x, i):
    """Insert x into linked list s at index i.

    >>> evens = Link(4, Link(2, Link(6)))
    >>> insert_link(evens, 8, 1)
    >>> insert_link(evens, 10, 4)
    >>> insert_link(evens, 12, 0)
    >>> insert_link(evens, 14, 10)
    Index out of range
    >>> print(evens)
    <12 4 8 2 6 10>
    """
    if s is Link.empty:
        print('Index out of range')
    elif i == 0:
        second = Link(s.first, s.rest)
        s.first = x
        s.rest = second
    elif i == 1 and s.rest is Link.empty:
        s.rest = Link(x)
    else:
        insert_link(s.rest, x, i-1)
```



这个需要自己注意，就是链表最后是否为空需要注意

6. 继承中需要注意的，如果改变子类中父类的变量，那么对父类本身是没有影响的

```
class A:
    x,y = 0,0
    def __init__(self):
        return
```

```
class B(A):
    def __init__(self):
        return
```

```
class C(A):
    def __init__(self):
        return
```

✓ 0.0s

```
B.x = 2
print(A.x, B.x, C.x)
```

✓ 0.0s

0 2 0

```
A.x += 1
print(A.x, B.x, C.x)
```

✓ 0.0s

1 2 1

7. 解决问题没有名为“urllib3.packages.six.moves”的模块

这个原因在于，运行CS61A是直接用自己的文件封装好的包来进行的，但是Python是调用自己的，所以就会出现，自己的包和环境没有任何问题，但是到了OK封闭有问题，这是因为Python版本和OK版本对不上，因此我下调了一下Python版本为3.7，可以正确导入包。

这个从另一方面说明了自己下的文件有问题，不知道是哪一年的文件了，逆天保存页。噫，原来是2023年的啊，气笑了。

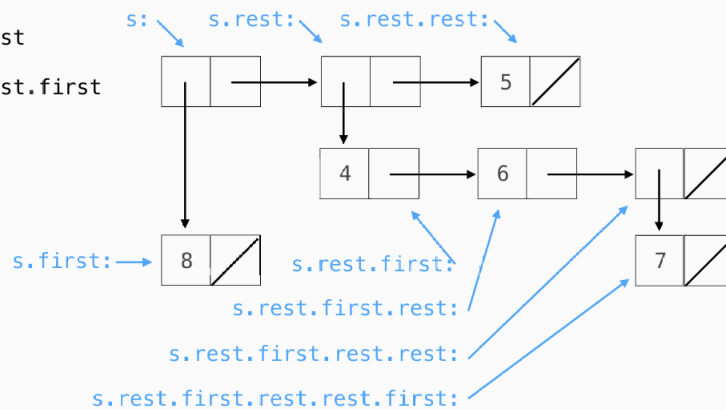
8. 调用父类函数

```
def withdraw(self, amount):
    self.withdrawals += 1
    fee = 0
    if self.withdrawals > self.free_withdrawals:
        fee = self.withdraw_fee
    return super().withdraw(amount + fee)
```

9. 新奇的链表表示方法

Nested Linked Lists

```
>>> s = Link(Link(8), Link(Link(4, Link(6, Link(Link(7))))), Link(5))
>>> print(s)
<<8> <4 6 <7>> 5>
>>> s.first.first
8
>>> s.rest.first.rest.rest.first
Link(7)
>>> s.rest.first.rest.rest.first.first
7
```



10. 以下是几个操作的说明

Assume that before each example below we execute:

`s = [2, 3]`

`t = [5, 6]`

Operation	Example	Result
append adds one element to a list	<code>s.append(t)</code> <code>t = 0</code>	<code>s → [2, 3, [5, 6]]</code> <code>t → 0</code>
extend adds all elements in one list to another list	<code>s.extend(t)</code> <code>t[1] = 0</code>	<code>s → [2, 3, 5, 6]</code> <code>t → [5, 0]</code>
addition & slicing create new lists containing existing elements	<code>a = s + [t]</code> <code>b = a[1:]</code> <code>a[1] = 9</code> <code>b[1][1] = 0</code>	<code>s → [2, 3]</code> <code>t → [5, 0]</code> <code>a → [2, 9, [5, 0]]</code> <code>b → [3, [5, 0]]</code>
The list function also creates a new list containing existing elements	<code>t = list(s)</code> <code>s[1] = 0</code>	<code>s → [2, 0]</code> <code>t → [2, 3]</code>
[...] creates a new list	<code>u = [s, t]</code>	<code>s → [2, 3]</code> <code>t → [5, 6]</code> <code>u → [[2, 3], [5, 6]]</code>

Operation	Example	Result
pop removes & returns the last element	<code>t = s.pop()</code>	<code>s → [2]</code> <code>t → 3</code>
remove removes the first element equal to the argument	<code>t.extend(t)</code> <code>t.remove(5)</code>	<code>s → [2, 3]</code> <code>t → [6, 5, 6]</code>

11. 这段代码很好地利用了树的结构性

A *hydra* is a Tree with a special structure. Each node has 0 or 2 children. All leaves are heads labeled 1. Each non-leaf body node is labeled with the number of leaves among its descendants.

Implement `chop_head(hydra, n)`, which takes a hydra and a positive integer `n`. It mutates `hydra` by chopping off the `n`th head from the left, which adds two new adjacent heads in its place. Update all ancestor labels.

```
def chop_head(hydra, n):
    assert n > 0 and n <= hydra.label
    if hydra.is_leaf():
        hydra.label = 2
        hydra.branches = [Tree(1), Tree(1)]
    else:
        hydra.label += 1
        left, right = hydra.branches
        if n > left.label:
            chop_head(right, n - left.label)
        else:
            chop_head(left, n)
```

